

Q1) Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:

1. Data type of all columns in the "customers" table.

```
select COLUMN_NAME, DATA_TYPE
from global-exchange-399511.target.INFORMATION_SCHEMA.COLUMNS
where table_name = 'customers'
```

Row	COLUMN_NAME	DATA_TYPE
1	customer_id	STRING
2	customer_unique_id	STRING
3	customer_zip_code_prefix	INT64
4	customer_city	STRING
5	customer_state	STRING

Actionable Insights: The output gives the column names and the datatype of each column in the customers table.

2. Get the time range between which the orders were placed.

```
select min(order_purchase_timestamp) as first_order,
max(order_purchase_timestamp) as last_order
from `target.orders`
```

Row	first_order	last_order
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC

Actionable Insights: The output shows the first order and last order places by the customers during the operations of Target in Brazil between 2016 and 2018.

3. Count the Cities & States of customers who ordered during the given period.

```
select count(distinct c.customer_city) as cities_count,
count(distinct c.customer_state) as states_count
from `target.customers` c inner join `target.orders` o
on c.customer_id = o.customer_id
```

Row	cities_count	states_count
1	4119	27

Actionable Insights: The output shows the unique total count of states and cities of the customers who ordered during the period.

Q2)In-depth Exploration:

1. Is there a growing trend in the no. of orders placed over the past years?

```
select extract(year from order_purchase_timestamp) as year, extract(month from
order_purchase_timestamp) as month, count(order_id) as order_count,
from `target.orders`
group by 1,2
order by 1,2
```

Row	year	month	no_of_orders
1	2016	9	4
2	2016	10	324
3	2016	12	1
4	2017	1	800
5	2017	2	1780
6	2017	3	2682
7	2017	4	2404
8	2017	5	3700
9	2017	6	3245
10	2017	7	4026

Actionable Insights: From the output we could see that the number of orders placed were small during months in the initial year 2016, and as the months passed we could see the sales started picking up and the number of orders gradually increased in the year 2017.

Recommendation: As we could see a growing trend in the number of orders placed over the years from 2016, the company could take it as a good sign and continue its operation in Brazil.

2. Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

```
select T2.month, round(avg((T1.order_count/T2.avg_count)),2) as
monthly_seasonality_index from
```

```

(select extract(year from order_purchase_timestamp) as year, extract(month from
order_purchase_timestamp) as month,
count(order_id) as order_count
from `target.orders` group by 1,2 order by 1,2) T1
inner join
(select A.year,A.month, avg(A.order_count) over(partition by A.year) as
avg_count from
(select distinct extract(year from order_purchase_timestamp) as year,
extract(month from order_purchase_timestamp) as month,
count(order_id) over(partition by extract(year from
order_purchase_timestamp),extract(month from order_purchase_timestamp)) as
order_count
from `target.orders` order by 1,2) A order by 1,2)T2
on T1.year = T2.year and T1.month = T2.month group by 1 order by 1

```

Row	month	monthly_seasonality_index
1	1	0.78
2	2	0.86
3	3	1.02
4	4	0.96
5	5	1.13
6	6	1.0
7	7	1.12
8	8	1.18
9	9	0.39
10	10	1.4
11	11	2.01
12	12	0.76

Actionable Insights: From the output, we can see that the months October and November have the most seasonality when compared to other months meaning more customers placed their order during these months.

Recommendation: Since the seasonality of October and November is high meaning more orders will be placed by the customers during these months, the business can come up with few strategies like i) keeping more number of products in stock during these months, ii) club products which are not sold most with other best selling products and provide best deals on them to attract more customers.

3. During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

- a. 0-6 hrs : Dawn
- b. 7-12 hrs : Mornings
- c. 13-18 hrs : Afternoon
- d. 19-23 hrs : Night

```
select A.Timings, count(A.order_id) as number_of_purchases from
(select *,
case
when extract(hour from order_purchase_timestamp) between 0 and 6 then 'Daws'
when extract(hour from order_purchase_timestamp) between 7 and 12 then
'Mornings'
when extract(hour from order_purchase_timestamp) between 13 and 18 then
'Afternoon'
when extract(hour from order_purchase_timestamp) between 19 and 23 then 'Night'
end as Timings
from `target.orders`) A
group by 1
order by 2 desc
```

Row	Timings	number_of_purchases
1	Afternoon	38135
2	Night	28331
3	Mornings	27733
4	Daws	5242

Actionable Insights: From the output we can see that during Afternoon time the Brazilian customers place the most orders.

Recommendation: Make sure that more workforce is present during the afternoon time to support supply and logistics.

Q3) Evolution of E-commerce orders in the Brazil region:

1. Get the month on month no. of orders placed in each state.

```
select distinct c.customer_state, extract(month from
o.order_purchase_timestamp) as month, count(o.order_id) as count_by_month
```

```

from `target.customers` c
inner join `target.orders` o
on c.customer_id = o.customer_id
group by 1,2
order by 1,2

```

Row	customer_state	month	count_by_month
1	AC	1	8
2	AC	2	6
3	AC	3	4
4	AC	4	9
5	AC	5	10
6	AC	6	7
7	AC	7	9
8	AC	8	7
9	AC	9	5
10	AC	10	6
11	AC	11	5

Actionable Insights: The output shows the number of orders placed in each state in each month by the customers.

2. How are the customers distributed across all the states?

```

select c.customer_state, count(c.customer_id) as count_of_people
from `target.customers` c inner join `target.orders` o on c.customer_id =
o.customer_id
group by 1
order by 2 desc

```

Row	customer_state	count_of_people
1	SP	41746
2	RJ	12852
3	MG	11635
4	RS	5466
5	PR	5045
6	SC	3637
7	BA	3380
8	DF	2140
9	ES	2033
10	GO	2020

Actionable Insights: The output shows the number of unique customers present in each state.

Q4) Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.

1. Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).

You can use the "payment_value" column in the payments table to get the cost of orders.

```
select B.year, B.cost_of_orders,
round((B.cost_of_orders-B.prev_val)/(B.prev_val)* 100,2) as percent_increase
from
(select A.year, A.cost_of_orders, lag(A.cost_of_orders) over(order by
A.cost_of_orders) as prev_val from
(select extract(year from o.order_purchase_timestamp) as
year,sum(p.payment_value) as cost_of_orders
from `target.orders` o inner join `target.payments` p
on o.order_id = p.order_id
where extract(month from o.order_purchase_timestamp) between 1 and 8
and extract(year from o.order_purchase_timestamp) in (2017, 2018)
group by 1
) A
order by 1) B
```

Row	year	cost_of_orders	percent_increase
1	2017	3669022.119999...	null
2	2018	8694733.839999...	136.98

Actionable Insights: The % increase in the cost of orders from year 2017 to 2018 (including months between Jan to Aug only) is 136.98.

Recommendation: From the %gain we see in cost of orders from 2017 to 2018, we can take this learnings and analyze and come up with new strategies in the coming years to boost the sales and gain profits.

2. Calculate the Total & Average value of order price for each state.

```
select c.customer_state, round(sum(ot.price),2) as total_price,
round(avg(ot.price),2) as average_price
from `target.customers` c inner join `target.orders` o
on c.customer_id = o.customer_id
inner join `target.order_items` ot on o.order_id = ot.order_id
group by 1
order by 2
```

Row	customer_state	total_price	average_price
1	RR	7829.43	150.57
2	AP	13474.3	164.32
3	AC	15982.95	173.73
4	AM	22356.84	135.5
5	RO	46140.64	165.97
6	TO	49621.74	157.53
7	SE	58920.85	153.04
8	AL	80314.81	180.89
9	RN	83034.98	156.97
10	PI	86914.08	160.36

Actionable Insights: The output shows the Total & Average value of order price for each state ordered by total price in ascending order.

3. Calculate the Total & Average value of order freight for each state.

```
select g.geolocation_state, round(sum(ot.freight_value),2) as
total_freight_value, round(avg(ot.freight_value),2) as average_freight_value
from `target.sellers` s inner join `target.order_items` ot
on s.seller_id = ot.seller_id
inner join `target.geolocation` g on s.seller_zip_code_prefix =
g.geolocation_zip_code_prefix
group by 1
order by 2
```

Row	geolocation_state	total_freight_value	average_freight_valu
1	PI	1773.28	36.94
2	AM	2208.6	27.27
3	AC	5385.76	32.84
4	SE	11798.2	29.13
5	RN	63552.26	15.93
6	PB	83959.72	34.69
7	RO	85745.36	50.32
8	MS	113813.87	25.98
9	CE	163715.97	54.44
10	MT	263738.15	32.07

Actionable Insights: The output shows the Total & Average value of order freight for each state. Ordered by total freight value in ascending order

Q5) Analysis based on sales, freight and delivery time.

1. Find the no. of days taken to deliver each order from the order's purchase date as delivery time.

Also, calculate the difference (in days) between the estimated & actual delivery date of an order.

Do this in a single query.

You can calculate the delivery time and the difference between the estimated & actual delivery date using the given formula:

- a. $\text{time_to_deliver} = \text{order_delivered_customer_date} - \text{order_purchase_timestamp}$
- b. $\text{diff_estimated_delivery} = \text{order_estimated_delivery_date} - \text{order_delivered_customer_date}$

```
select order_id,order_purchase_timestamp, order_delivered_customer_date,
order_estimated_delivery_date,
date_diff(date(order_delivered_customer_date),date(order_purchase_timestamp),day) as time_to_deliver,
date_diff(date(order_estimated_delivery_date),date(order_delivered_customer_date),day) as diff_estimated_delivery
from `target.orders`
where order_delivered_customer_date is not null
```

Row	order_id	order_purchase_timestamp	order_delivered_customer_date	order_estimated_delivery_date	time_to_deliver	diff_estimated_delivery
1	1950d777989f6a877539f53795b4c3c3	2018-02-19 19:48:52 UTC	2018-03-21 22:03:51 UTC	2018-03-09 00:00:00 UTC	30	-12
2	2c45c33d2f9cb8ff8b1c86cc28c11c30	2016-10-09 15:39:56 UTC	2016-11-09 14:53:50 UTC	2016-12-08 00:00:00 UTC	31	29
3	65d1e226dfaeb8cdc42f665422522d14	2016-10-03 21:01:41 UTC	2016-11-08 10:58:34 UTC	2016-11-25 00:00:00 UTC	36	17
4	635c894d068ac37e6e03dc54eccb6189	2017-04-15 15:37:38 UTC	2017-05-16 14:49:55 UTC	2017-05-18 00:00:00 UTC	31	2
5	3b97562c3aee8bdcdb5c2e45a50d5e1	2017-04-14 22:21:54 UTC	2017-05-17 10:52:15 UTC	2017-05-18 00:00:00 UTC	33	1
6	68f47f50f04c4cb6774570cfde3a9aa7	2017-04-16 14:56:13 UTC	2017-05-16 09:07:47 UTC	2017-05-18 00:00:00 UTC	30	2
7	276e9ec344d3bf029ff83a161c6b3ce9	2017-04-08 21:20:24 UTC	2017-05-22 14:11:31 UTC	2017-05-18 00:00:00 UTC	44	-4
8	54e1a3c2b97fb0809da548a59f64c813	2017-04-11 19:49:45 UTC	2017-05-22 16:18:42 UTC	2017-05-18 00:00:00 UTC	41	-4
9	fd04fa4105ee8045f6a0139ca5b49f27	2017-04-12 12:17:08 UTC	2017-05-19 13:44:52 UTC	2017-05-18 00:00:00 UTC	37	-1
10	302bb8109d097a9fc6e9cef5917d1f3	2017-04-19 22:52:59 UTC	2017-05-23 14:19:48 UTC	2017-05-18 00:00:00 UTC	34	-5

Actionable Insights: In the output, the 'time_to_deliver' column shows the number of days taken to deliver the order from the day the customer placed it.

The 'diff_estimated_delivery' column shows the difference between the estimated & actual delivery date. The negative value means the order delivery to the customer was delayed when compared to the estimated date. The positive value means the order was delivered to the customer very early when compared with the estimated date. The value 0 means the order was delivered on the day it is expected to be delivered.

2. Find out the top 5 states with the highest & lowest average freight value.

```
with final as(
select g.geolocation_state, round(avg(ot.freight_value),2) as
average_freight_value
from `target.sellers` s inner join `target.order_items` ot
on s.seller_id = ot.seller_id
inner join `target.geolocation` g on s.seller_zip_code_prefix =
g.geolocation_zip_code_prefix
group by 1
order by 2 desc)

(select * from final limit 5)
union all
(select * from final order by 2 limit 5)
```

Row	geolocation_state	average_freight_valu
1	CE	54.44
2	RO	50.32
3	PI	36.94
4	PB	34.69
5	AC	32.84
6	RN	15.93
7	SP	18.44
8	RJ	18.93
9	DF	18.99
10	PR	22.11

Actionable Insights: The output shows the top 5 states arranged in decreasing order of the average freight value and the bottom 5 states arranged in increasing order of the average freight value.

3. Find out the top 5 states with the highest & lowest average delivery time.

```
with final as(
select c.customer_state,
round(avg(date_diff(date(o.order_delivered_customer_date),date(o.order_purchase_timestamp),day)),2) as average_delivery_time
from `target.customers` c inner join `target.orders` o
on c.customer_id = o.customer_id
group by 1
order by 2 desc)
```

```
(select * from final limit 5)
union all
(select * from final order by 2 limit 5)
```

Row	customer_state	average_delivery_time
1	RR	29.34
2	AP	27.18
3	AM	26.36
4	AL	24.5
5	PA	23.73
6	SP	8.7
7	PR	11.94
8	MG	11.95
9	DF	12.9
10	SC	14.91

Actionable Insights: The output shows the top 5 states arranged in decreasing order of the average delivery time and the bottom 5 states arranged in increasing order of the average delivery time.

4. Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery. You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

```

with final as
(
select *, dense_rank() over(order by A.diff_actual_estimated) as ranking from
(select c.customer_state,
round(avg(date_diff(date(o.order_estimated_delivery_date),date(o.order_delivered_customer_date),day)),2) as diff_actual_estimated
from `target.customers` c inner join `target.orders` o
on c.customer_id = o.customer_id
where o.order_delivered_customer_date is not null
group by 1) A
order by ranking
)
(select * from final
where final.ranking <= 5)

```

Row	customer_state	diff_actual_estimated	ranking
1	AL	8.71	1
2	MA	9.57	2
3	SE	10.02	3
4	ES	10.5	4
5	BA	10.79	5

Actionable Insights: In the output, ranking 1 to 5 shows the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.

Q6) Analysis based on the payments:

1. Find the month on month no. of orders placed using different payment types.

```

select extract(year from o.order_purchase_timestamp) as year, extract(month
from o.order_purchase_timestamp) as month, p.payment_type, count(o.order_id) as
no_of_orders
from `target.customers` c inner join `target.orders` o
on c.customer_id = o.customer_id
inner join `target.payments` p on
o.order_id = p.order_id
group by 1,2,3
order by 1,2,3

```

Row	year	month	payment_type	no_of_orders
1	2016	9	credit_card	3
2	2016	10	UPI	63
3	2016	10	credit_card	254
4	2016	10	debit_card	2
5	2016	10	voucher	23
6	2016	12	credit_card	1
7	2017	1	UPI	197
8	2017	1	credit_card	583
9	2017	1	debit_card	9
10	2017	1	voucher	61

Actionable Insights: The output shows the month on month no. of orders placed between 2016 and 2018 using different payment types.

Recommendation: From the output, the company can analyze different payment methods used to place an order and take actions like providing best offers on certain payment methods to increase the sales and to attract more customers in buying the products.

2. Find the no. of orders placed on the basis of the payment installments that have been paid.

```
select p.payment_installments, count(o.order_id) as no_of_orders
from `target.customers` c inner join `target.orders` o
on c.customer_id = o.customer_id
inner join `target.payments` p on
o.order_id = p.order_id
where p.payment_value > 0
group by 1
order by 1
```

Row	payment_installment	no_of_orders
1	0	2
2	1	52537
3	2	12413
4	3	10461
5	4	7098
6	5	5239
7	6	3920
8	7	1626
9	8	4268
10	9	644

Actionable Insights: The output shows the number of orders placed on the basis of the payment installments that have been paid.